# INTERNSHIP PROJECT REPORT

*on*

# AI MULTI-OBJECT DETECTION SYSTEM

*Real-Time Object Detection, Helmet Recognition & Person Counting*

Submitted by

**Shivam Rai**

Enrollment No.: EN23CS301969

B.Tech — Computer Science & Engineering

Medi-Caps University, Indore

**Organization:** Hindalco Industries Limited Mahan

**Training Head :** Mr. Mohit Shukla

**Project Guide:** Mr. Mithlesh Nirmalkar

**Internship Duration:** 08/06/2026 to 03/07/2026

**Department of Computer Science & Engineering**

**MEDI-CAPS UNIVERSITY, INDORE — 453331 | JUNE 2026**

# TABLE OF CONTENTS

# CERTIFICATE

This is to certify that Mr. Shivam Rai, Enrollment No. EN23CS301969, a student of B.Tech Computer Science & Engineering at Medi-Caps University, Indore, has successfully completed his internship at Hindalco Industries Limited Mahan from 08th June 2026 to 03rd July 2026.

During this internship, he worked on the project titled "AI Multi-Object Detection System" under the guidance of Mr. Mithlesh Nirmalkar (Project Guide). The project involved developing a real-time multi-object detection and helmet recognition system using Python, OpenCV, YOLOv8, and PyTorch.

His work reflects dedication, technical competence, and a strong understanding of artificial intelligence and computer vision technologies. We wish him success in his future academic and professional endeavours.

**Mr. Mohit Shukla**
Training Head
Hindalco Industries Limited Mahan

**Mr. Mithlesh Nirmalkar**
Project Guide
Hindalco Industries Limited Mahan

*Seal & Signature of Organization*

# DECLARATION

I, Shivam Rai (Enrollment No. EN23CS301969), student of B.Tech Computer Science & Engineering at Medi-Caps University, Indore, hereby declare that this internship report titled "AI Multi-Object Detection System" is my own original work carried out at Hindalco Industries Limited Mahan under the supervision of Mr. Mithlesh Nirmalkar.

I further declare that this report has not been submitted to any other university or institution for the award of any degree, diploma, or certificate. All sources of information used in this report have been duly acknowledged in the references section.

The project implementation, code, and analysis presented in this report are carried out independently by me as part of my internship training.

**Shivam Rai**
Enrollment No.: EN23CS301969
B.Tech CSE, Medi-Caps University, Indore
Date: _____

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Hindalco Industries Limited for providing me with the invaluable opportunity to undertake this internship. The exposure to a real industrial environment has significantly enhanced my technical knowledge and professional outlook.

I am deeply thankful to my Training Head, Mr. Mohit Sir, for his constant encouragement, support, and motivating guidance throughout the duration of the internship. His insights into the industrial application of AI technologies helped me approach the project with clarity and confidence.

I extend my heartfelt gratitude to my Project Guide, Mithlesh Sir, for his exceptional technical mentorship, constructive feedback, and patient guidance during every phase of the project development. His expertise in computer vision and deep learning was instrumental in shaping this project.

I am also grateful to the entire technical team at Hindalco Industries Limited for their cooperation and willingness to share their knowledge and experience. Their practical perspectives on industrial safety and automation enriched my understanding of the domain.

I would like to thank Prof. Mr. Kailash Chandra Bandhu, Head of the Department of Computer Science & Engineering, Medi-Caps University, Indore, and all the faculty members who equipped me with the foundational knowledge necessary to undertake this project successfully.

Finally, I am grateful to my family and friends for their unwavering moral support and encouragement throughout this journey.

**Shivam Rai**
EN23CS301969
B.Tech CSE — III Year
Medi-Caps University, Indore

# ABSTRACT

The rapid advancement of deep learning and computer vision has opened transformative possibilities in industrial safety, surveillance, and automation. This internship project, developed at Hindalco Industries Limited, presents a comprehensive AI Multi-Object Detection System designed to address one of the most pressing challenges in industrial environments — the real-time monitoring of workers and their compliance with safety protocols.

The system is built upon the YOLOv8 (You Only Look Once, version 8) architecture, which represents the current state of the art in real-time object detection. Two distinct detection pipelines are integrated: a pre-trained YOLOv8 model operating on the COCO dataset for general multi-object detection, and a custom-trained YOLOv8 model specifically fine-tuned for the detection of safety helmets in industrial contexts.

The backend of the system is developed entirely in Python, leveraging OpenCV for video capture and visualization, Ultralytics for YOLOv8 model inference, PyTorch as the deep learning framework, and NumPy for numerical computation. The system processes frames captured from a webcam or video file in real time, annotating each frame with bounding boxes, class labels, and confidence scores.

Beyond static detection, the system incorporates object tracking using the SORT (Simple Online and Realtime Tracking) algorithm, which assigns unique IDs to detected objects across frames, enabling continuous monitoring of individual entities. A virtual counting line is implemented, allowing the system to count the number of persons who cross a predefined boundary within the frame — a feature directly applicable to monitoring entry and exit points in industrial facilities.

Performance metrics including Frames Per Second (FPS) are computed and displayed in real time, providing a direct assessment of system throughput. The application also supports video recording, allowing processed output to be saved for post-analysis and evidence documentation. Training metrics including Precision, Recall, mean Average Precision at IoU threshold 50 (mAP50), and mAP50-95 were evaluated on the custom helmet dataset to establish the effectiveness of the model.

The system was trained and tested on a machine equipped with an AMD Ryzen 5 processor and an NVIDIA GeForce RTX 3050 GPU, with 16 GB RAM. GPU acceleration through CUDA significantly reduced inference times, enabling real-time operation at satisfactory frame rates.

The results demonstrate that the system achieves reliable detection performance and can serve as a scalable, cost-effective solution for automated industrial safety monitoring. Future extensions include cloud dashboard integration, PPE detection beyond helmets, and automated alert generation upon detection of safety violations.

***Keywords:*** *YOLOv8, Object Detection, Computer Vision, Helmet Detection, OpenCV, PyTorch, SORT Tracking, Person Counting, Industrial Safety, Deep Learning, Real-Time Processing*

# ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| CNN | Convolutional Neural Network |
| COCO | Common Objects in Context |
| CUDA | Compute Unified Device Architecture |
| CV | Computer Vision |
| DNN | Deep Neural Network |
| FPS | Frames Per Second |
| GPU | Graphics Processing Unit |
| IDE | Integrated Development Environment |
| IoU | Intersection over Union |
| JSON | JavaScript Object Notation |
| mAP | Mean Average Precision |
| ML | Machine Learning |
| NMS | Non-Maximum Suppression |
| PPE | Personal Protective Equipment |
| RAM | Random Access Memory |
| ReLU | Rectified Linear Unit |
| RGB | Red Green Blue |
| SORT | Simple Online and Realtime Tracking |
| SSD | Solid State Drive |
| UI | User Interface |
| YOLO | You Only Look Once |

# Chapter 1: Introduction

## 1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) refers to the simulation of human cognitive processes by computational systems. These processes include learning from experience, recognizing patterns, making decisions, and solving complex problems. Since the mid-twentieth century, AI has evolved from theoretical frameworks to practical systems that power everything from search engines and recommendation algorithms to autonomous vehicles and medical diagnostics.

In the context of industrial applications, AI has emerged as a critical enabler of operational efficiency, quality control, and safety monitoring. Industrial environments are complex, dynamic, and often hazardous. Traditional monitoring methods — primarily human surveillance and periodic manual inspections — are inherently limited in scale, consistency, and speed. AI-driven automation addresses these limitations by enabling continuous, objective, and high-speed monitoring of industrial processes and personnel.

## 1.2 Machine Learning and Deep Learning

Machine Learning (ML) is a subfield of AI that focuses on algorithms capable of learning from data without being explicitly programmed. Rather than relying on hand-coded rules, ML algorithms infer patterns and relationships from training examples. Supervised learning, unsupervised learning, and reinforcement learning are the three primary paradigms, each suited to different categories of problems.

Deep Learning is a specialized branch of machine learning that employs artificial neural networks with many layers — hence the term 'deep' — to model and learn complex, hierarchical representations of data. Deep learning has achieved remarkable breakthroughs in image recognition, natural language processing, speech synthesis, and many other domains. The availability of large datasets, powerful GPUs, and improved training algorithms has been central to the rapid advancement of deep learning over the past decade.

## 1.3 Computer Vision

Computer Vision is a field of AI that enables machines to interpret and understand visual information from the world. Just as humans use their eyes and brain to process images and extract meaning, computer vision systems use cameras and algorithms to analyze digital images and

videos. Core tasks in computer vision include image classification, object detection, image segmentation, pose estimation, and optical flow analysis.

Object detection is a particularly important task that involves identifying the presence and location of specific objects within an image or video frame. An object detection system must not only classify what objects are present but also precisely localize them using bounding boxes. Applications of object detection span autonomous driving, medical imaging, retail analytics, security surveillance, and industrial safety monitoring.

## 1.4 YOLO: You Only Look Once

The YOLO (You Only Look Once) family of object detection architectures was introduced by Redmon et al. in 2016 and has since become the benchmark for real-time object detection. Unlike earlier region-proposal-based detectors such as R-CNN and Faster R-CNN, which process candidate regions sequentially, YOLO frames detection as a single regression problem: given an entire image as input, YOLO directly predicts bounding box coordinates and class probabilities in a single forward pass through the network. This design choice makes YOLO dramatically faster than its predecessors while maintaining competitive accuracy.

The YOLO architecture divides the input image into an S x S grid. Each grid cell predicts B bounding boxes and their associated confidence scores, as well as C class probabilities. At inference time, Non-Maximum Suppression (NMS) is applied to eliminate redundant detections and retain only the most confident bounding box for each object. Subsequent versions of YOLO — including YOLOv2, YOLOv3, YOLOv4, YOLOv5, YOLOv7, and YOLOv8 — have progressively improved detection speed, accuracy, and versatility.

## 1.5 YOLOv8

YOLOv8, developed by Ultralytics and released in January 2023, represents the current state of the art in the YOLO family. It introduces a new backbone architecture, an anchor-free detection head, and a decoupled head design that separates classification and regression tasks. These improvements result in higher accuracy across a range of object scales while maintaining the real-time speed that YOLO is known for.

YOLOv8 is available in five sizes — Nano (n), Small (s), Medium (m), Large (l), and Extra-large (x) — enabling deployment across a wide spectrum of hardware, from embedded devices to high-performance GPU servers. The Ultralytics library provides a clean Python API

for training, validation, and inference, significantly lowering the barrier to entry for practitioners building custom detection systems.

## 1.6 Industrial Safety and Helmet Detection

Workplace safety is a fundamental concern in industrial environments such as manufacturing plants, construction sites, and mining facilities. Falls from height and head injuries due to falling objects are among the leading causes of fatalities and serious injuries in these settings. The mandatory use of safety helmets is a universally enforced regulation in such environments, yet ensuring compliance across large and complex facilities remains a significant operational challenge.

Automated helmet detection using computer vision offers a practical and scalable solution to this problem. A custom-trained YOLOv8 model can be deployed on existing camera infrastructure to monitor workers in real time, generating alerts when non-compliance is detected. This reduces the burden on human safety officers and enables consistent, 24-hour monitoring coverage.

## 1.7 Literature Review

The field of object detection has a rich research history. Below is a review of key contributions that inform the development of this project:

| S.No. | Author(s) | Year | Contribution |
|---|---|---|---|
| 1 | Redmon et al. | 2016 | Introduced YOLO, the first single-pass object detector, enabling real-time detection. |
| 2 | Redmon & Farhadi | 2017 | YOLOv2 improved accuracy with batch normalization, anchor boxes, and multi-scale training. |
| 3 | Redmon & Farhadi | 2018 | YOLOv3 used Darknet-53 backbone and multi-scale predictions for improved small object detection. |
| 4 | Bochkovskiy et al. | 2020 | YOLOv4 introduced CSP networks, PANet, and advanced augmentation strategies for superior performance. |

| | | | |
|---|---|---|---|
| 5 | Jocher et al. | 2020 | YOLOv5 provided a PyTorch implementation with streamlined training and deployment pipelines. |
| 6 | Wang et al. | 2022 | YOLOv7 achieved state-of-the-art performance through trainable bag-of-freebies strategies. |
| 7 | Jocher et al. | 2023 | YOLOv8 introduced anchor-free heads and decoupled classification/regression for improved accuracy. |
| 8 | Bewley et al. | 2016 | SORT algorithm proposed a simple, efficient approach to online multi-object tracking using Kalman filters. |
| 9 | He et al. | 2016 | ResNet introduced residual connections, enabling training of very deep networks without degradation. |
| 10 | Bradski | 2000 | OpenCV established as the foundational library for real-time computer vision applications. |
| 11 | LeCun et al. | 1998 | Introduced Convolutional Neural Networks, laying the groundwork for modern computer vision. |
| 12 | Lin et al. | 2014 | COCO dataset introduced as a comprehensive benchmark for object detection and segmentation. |
| 13 | Girshick et al. | 2014 | R-CNN pioneered region-based detection, separating proposal generation from classification. |
| 14 | Ren et al. | 2015 | Faster R-CNN unified region proposal and detection in a single neural network. |
| 15 | Liu et al. | 2016 | SSD introduced multi-scale feature maps for detection, offering a balance of speed and accuracy. |
| 16 | He et al. | 2017 | Mask R-CNN extended Faster R-CNN with instance segmentation capability. |

***Table 1.1: Literature Review Summary***

## 1.8 Problem Statement

Manual monitoring of large industrial areas is time-consuming, resource-intensive, and prone to human fatigue and error. Ensuring worker compliance with safety protocols such as

helmet-wearing across expansive factory floors or construction sites is practically impossible through manual supervision alone. Simultaneously, tracking multiple moving objects, monitoring access control through counting, and maintaining a continuous video record for audit purposes compound the operational challenge.

An automated AI-based system capable of performing real-time multi-object detection, safety helmet recognition, object tracking, and person counting addresses these challenges in a unified, cost-effective, and scalable manner.

## 1.9 Objectives

The following objectives were established for this project:

1. Develop a real-time object detection system using YOLOv8 and the COCO dataset capable of identifying and localizing 80 object classes simultaneously from live webcam or video input.
2. Train a custom YOLOv8 model on a helmet-specific annotated dataset to detect the presence or absence of safety helmets on workers in industrial video streams.
3. Implement object tracking using the SORT algorithm to maintain unique identification of detected objects across consecutive video frames.
4. Design and implement a virtual counting line mechanism to accurately count persons crossing a predefined boundary within the video frame.
5. Compute and display real-time performance metrics including Frames Per Second (FPS) and detection confidence scores.
6. Implement video recording capability to save annotated output video for documentation and post-analysis purposes.
7. Evaluate the custom helmet detection model using standard metrics including Precision, Recall, mAP50, and mAP50-95.

## 1.10 Significance of the Project

This project holds significant value across multiple dimensions. From an industrial safety perspective, it provides a scalable, automated solution to enforce helmet compliance and monitor worker movement without requiring dedicated human monitors. From a technical perspective, it demonstrates the practical integration of a state-of-the-art detection architecture with tracking, counting, and recording capabilities into a cohesive real-time system.

From an academic perspective, the project develops deep expertise in computer vision, deep learning, dataset preparation, model training, and system integration — competencies that are directly aligned with current industry demands. The project was executed in a real industrial environment at Hindalco Industries Limited, ensuring that the outcomes have direct practical relevance.

# Chapter 2: Company Profile

## 2.1 About Hindalco Industries Limited Mahan

Hindalco Industries Limited Mahan is one of India's largest and most diversified metals and mining companies, and a flagship entity of the Aditya Birla Group — a global conglomerate with operations spanning 36 countries. Hindalco is a global leader in aluminium production and is among the largest copper producers in Asia. The company's operations span the entire value chain, from bauxite mining and alumina refining to aluminium smelting, rolling, and fabrication, as well as copper smelting and refining.

Founded in 1958 and headquartered in Mumbai, Hindalco has grown into a Fortune 500 company with consolidated revenues exceeding USD 21 billion. Its global footprint includes manufacturing plants in India, the United States, Europe, South America, and Asia-Pacific, employing over 60,000 people worldwide. In India, Hindalco operates major aluminium plants at Renukoot, Belur, Taloja, Alupuram, and the Mahan Aluminium complex in Singrauli, Madhya Pradesh.

## 2.2 Technology and Innovation at Hindalco

Hindalco has consistently positioned itself as a technology-driven organization committed to continuous innovation and operational excellence. The company has made significant investments in Industry 4.0 technologies, including automation, the Industrial Internet of Things (IIoT), data analytics, and artificial intelligence, to enhance productivity, quality, and safety across its manufacturing operations.

At the Mahan Aluminium plant and other facilities, digital transformation initiatives have resulted in the deployment of advanced sensor networks, predictive maintenance systems, energy optimization algorithms, and computer vision-based quality inspection platforms. These technologies have enabled Hindalco to achieve significant reductions in operational costs, energy consumption, and workplace incidents.

## 2.3 Workplace Safety at Hindalco

Safety is a core value at Hindalco Industries Limited, embedded in the company's culture and operational practices at every level. The company's safety management system is aligned with international standards including ISO 45001 and OHSAS 18001. Personal Protective

Equipment (PPE) compliance, including the mandatory wearing of safety helmets in designated areas, is rigorously enforced.

The deployment of AI-based monitoring systems to supplement manual safety inspections represents a strategic initiative aligned with Hindalco's broader digital transformation agenda. The AI Multi-Object Detection System developed during this internship is directly relevant to this initiative, providing automated, real-time helmet compliance monitoring and movement tracking capabilities.

## 2.4 Internship Context

This internship was undertaken at Hindalco Industries Limited over a period of four weeks, from 8th June 2026 to 3rd July 2026. The internship provided direct exposure to industrial computing environments, safety-critical application development, and the practical challenges of deploying AI solutions in real operational contexts. The project was defined and supervised by the technical team at Hindalco, with the objective of developing a functional prototype of an AI-based safety monitoring system.

# Chapter 3: Requirements Specification

## 3.1 User Characteristics

The primary users of this system are industrial safety officers, plant managers, and security supervisors responsible for monitoring personnel in manufacturing or construction environments. These users do not require deep technical knowledge of AI or computer vision to operate the system — the interface is designed to be straightforward and interpretable. Secondary users include IT administrators who manage system installation and maintenance, and data analysts who review recorded video logs and performance reports.

## 3.2 Functional Requirements

- FR1: The system shall process real-time video input from a webcam or pre-recorded video file at a minimum of 15 FPS under normal operating conditions.
- FR2: The system shall detect and localize multiple object classes from the COCO dataset (80 classes) simultaneously in each video frame using YOLOv8.
- FR3: The system shall detect the presence or absence of safety helmets on persons in the video frame using a custom-trained YOLOv8 model.
- FR4: The system shall assign and maintain unique tracking IDs to detected objects across consecutive frames using the SORT algorithm.
- FR5: The system shall increment a person counter when a tracked individual crosses a predefined virtual counting line within the frame.
- FR6: The system shall display real-time annotations on the video feed, including bounding boxes, class labels, confidence scores, and tracking IDs.
- FR7: The system shall display the current FPS value on the video feed in real time.
- FR8: The system shall provide functionality to record the annotated video output to a file for post-analysis and documentation.
- FR9: The system shall gracefully handle scenarios where no objects are detected in a frame without interrupting the video pipeline.

## 3.3 Non-Functional Requirements

- NFR1 — Performance: The system shall achieve inference latency of less than 100 milliseconds per frame on GPU hardware, enabling real-time operation.

- NFR2 — Accuracy: The custom helmet detection model shall achieve a minimum mAP50 of 85% on the validation dataset.
- NFR3 — Reliability: The system shall operate continuously for extended periods without memory leaks or performance degradation.
- NFR4 — Usability: All visual annotations shall be clearly legible with sufficient contrast against diverse video backgrounds.
- NFR5 — Maintainability: The codebase shall be structured in modular components with descriptive variable names and comments for ease of maintenance and extension.
- NFR6 — Portability: The system shall be deployable on any Windows or Linux system equipped with a compatible GPU and Python 3.8 or higher.

## 3.4 Software Requirements

| Software/Library | Version | Purpose |
|---|---|---|
| Python | 3.x | Core programming language |
| Ultralytics YOLOv8 | 8.x | Object detection and training framework |
| OpenCV (cv2) | 4.x | Video capture, display, and image processing |
| PyTorch | 2.x | Deep learning backend for YOLOv8 |
| NumPy | 1.24+ | Numerical array operations |
| Matplotlib | 3.x | Training curve visualization |
| SciPy | 1.x | SORT tracking algorithm (linear assignment) |
| FilterPy | 1.4+ | Kalman filter implementation for SORT |
| Visual Studio Code | Latest | Primary IDE |
| Git | 2.x | Version control |
| Windows 11 / Ubuntu 22.04 | — | Operating system |

***Table 3.1: Software Requirements***

## 3.5 Hardware Requirements

| Component | Minimum | Recommended (Used) |
|---|---|---|
| Processor | Intel Core i5 / AMD Ryzen 5 | AMD Ryzen 5 (Used) |

| | | |
|---|---|---|
| GPU | NVIDIA GTX 1060 4GB | NVIDIA RTX 3050 4GB VRAM (Used) |
| RAM | 8 GB | 16 GB (Used) |
| Storage | 5 GB free | SSD, 512 GB (Used) |
| Camera | 720p USB Webcam | Built-in HD Webcam (Used) |
| CUDA | 10.2+ | CUDA 11.8 (Used) |

***Table 3.2: Hardware Requirements***

## 3.6 Constraints and Assumptions

The following constraints and assumptions govern the design and operation of the system:

- The system is primarily designed for indoor industrial environments with adequate, consistent lighting. Detection accuracy may degrade under extreme low-light conditions or direct glare.
- The custom helmet detection model is trained on a specific dataset and may require retraining or fine-tuning if deployed in environments with significantly different helmet styles, colors, or imaging conditions.
- Real-time performance at high frame rates requires a CUDA-compatible NVIDIA GPU. CPU-only operation is supported but will result in reduced frame rates.
- The virtual counting line is a fixed configuration parameter and must be manually adjusted if the camera position or field of view changes.
- The system assumes that the input video stream provides sufficient resolution (minimum 480p) for reliable face and helmet detection.
- Network connectivity is not required for local operation; cloud integration features described in Future Scope will require network access.

# Chapter 4: System Design

## 4.1 System Architecture

The AI Multi-Object Detection System is organized into a layered architecture comprising four primary components: the Input Layer, the AI Processing Layer, the Tracking and Analytics Layer, and the Output Layer. This modular design ensures that each component can be developed, tested, and upgraded independently without affecting the rest of the pipeline.

The Input Layer is responsible for acquiring raw video frames from the camera or video file. The AI Processing Layer applies the YOLOv8 models to each frame to produce detection results. The Tracking and Analytics Layer applies the SORT algorithm, maintains the person counter, and computes FPS. The Output Layer renders annotations on frames and delivers the final output to the display and/or video file.

***Figure 4.1.1: High-Level System Architecture Block Diagram***

## 4.2 Module Descriptions

### 4.2.1 Video Capture Module

The Video Capture Module wraps OpenCV's VideoCapture class to initialize and manage the camera or video file input stream. It handles frame acquisition, resolution configuration, and graceful release of the capture resource upon application termination. The module is designed to accept either a camera index (integer) or a file path (string) as input, providing flexibility between live and recorded video sources.

### 4.2.2 Object Detection Module — COCO Model

The COCO Detection Module loads a pre-trained YOLOv8 model (yolov8n.pt or yolov8m.pt) trained on the COCO dataset, which encompasses 80 object categories including person, car, bicycle, chair, laptop, and many others. For each frame, the module performs inference and returns a list of Detection objects, each containing a bounding box (x1, y1, x2, y2), a class index, a class label string, and a confidence score. A confidence threshold of 0.40 is applied to filter low-confidence detections.

### 4.2.3 Helmet Detection Module — Custom Model

The Helmet Detection Module loads the custom-trained YOLOv8 model fine-tuned on the helmet dataset. The model outputs bounding boxes labeled 'helmet' or 'no_helmet', enabling

the system to visually distinguish compliant workers from non-compliant ones. Bounding boxes for 'helmet' detections are rendered in green, while 'no_helmet' detections are rendered in red, providing an immediate visual cue to safety officers monitoring the output.

### 4.2.4 Object Tracking Module — SORT

The SORT (Simple Online and Realtime Tracking) algorithm is implemented as the object tracking module. SORT uses a Kalman filter to predict the next position of each tracked object based on its previous trajectory. Detected bounding boxes are associated with existing tracks using the Hungarian algorithm (linear sum assignment), which minimizes the total IoU-based cost between predicted and detected positions. Each successfully matched object is assigned a persistent integer tracking ID. New tracks are initialized for unmatched detections, and tracks that have not been matched for a configurable number of frames are terminated.

### 4.2.5 Person Counting Module

The Person Counting Module implements a virtual counting line as a horizontal or diagonal line drawn across the video frame at a user-configurable position. When the centroid of a tracked person's bounding box crosses the line from one side to the other, the person counter is incremented. The direction of crossing is also tracked, enabling separate counts for persons entering and exiting the monitored zone. The counter value is rendered as an on-screen text overlay in the upper portion of the frame.

### 4.2.6 Performance Monitoring Module

The Performance Monitoring Module computes the real-time FPS of the detection pipeline by measuring the elapsed time between consecutive frame processing operations using Python's time module. A rolling average over the last ten frames is computed to smooth transient fluctuations and provide a stable FPS readout. The FPS value is rendered on the video frame as an on-screen overlay.

### 4.2.7 Video Recording Module

The Video Recording Module uses OpenCV's VideoWriter class to encode and write annotated frames to an output video file in the MP4 format (using the mp4v codec). The output file is named with a timestamp prefix to prevent overwriting of previous recordings. Recording can be toggled on and off by the user via keyboard input during runtime. The module correctly initializes the VideoWriter with the same frame dimensions and FPS as the input source.

## 4.3 Data Flow

The data flow through the system follows a sequential pipeline. A raw video frame is acquired from the capture source. The frame is passed to the COCO detection model and simultaneously to the helmet detection model. Both models produce detection results, which are merged. The merged detections are passed to the SORT tracker, which updates tracking IDs. The person counting module checks for line crossings based on updated centroid positions. All annotations — bounding boxes, labels, confidence scores, tracking IDs, FPS, and person count — are rendered on a copy of the original frame. The annotated frame is displayed and, if recording is active, written to the output file.

***Figure 4.3.1: Data Flow Diagram — AI Multi-Object Detection Pipeline***

## 4.4 Database Design

The current implementation uses a lightweight, file-based data storage approach rather than a relational database. Detection metadata — including timestamps, detected class labels, confidence scores, tracking IDs, and frame indices — are written to a JSON log file (detections_log.json) at configurable intervals. This log file serves as the audit trail for post-analysis and reporting. The JSON schema stores each detection event as an object with keys: frame_id, timestamp, objects (array of detection records), person_count, and fps.

For future production deployment with real-time dashboards and multi-camera integration, migration to a time-series database such as InfluxDB or a document store such as MongoDB is recommended, as these are optimized for the high-throughput, append-heavy write patterns characteristic of video analytics applications.

## 4.5 UML Diagrams

### 4.5.1 Use Case Diagram

The primary actor in the system is the Safety Officer, who interacts with the system through the following use cases: Start Video Feed, View Real-Time Detections, View Helmet Compliance Status, View Person Count, Toggle Video Recording, and Stop Video Feed. A secondary actor, the System Administrator, interacts with the use cases: Configure Camera Source, Configure Counting Line, Load Detection Models, and Review Detection Logs.

***Figure 4.5.1: Use Case Diagram — AI Multi-Object Detection System***

### 4.5.2 Sequence Diagram

The sequence diagram illustrates the message flow during a single frame processing cycle: (1) VideoCapture provides a frame to the Main Controller. (2) Main Controller sends the

frame to YOLOv8COCOModel and YOLOv8HelmetModel concurrently. (3) Both models return detection lists. (4) Main Controller merges detections and sends to SORT Tracker. (5) SORT Tracker returns updated tracks with IDs. (6) Person Counter checks for line crossings and updates count. (7) AnnotationRenderer draws all overlays on the frame. (8) DisplayModule renders the annotated frame. (9) If recording is active, VideoWriter writes the frame to file.

***Figure 4.5.2: Sequence Diagram — Frame Processing Cycle***

### 4.5.3 Activity Diagram

The activity diagram for the overall system begins at system startup: models are loaded, VideoCapture is initialized, and the main loop begins. In each iteration: a frame is read; if no frame is available, the loop terminates. Otherwise, COCO and helmet detections are run in parallel. Detections are tracked; line crossing is checked; annotations are applied; the frame is displayed. If the user presses 'r', recording is toggled. If the user presses 'q', the loop terminates. On termination, resources are released and logs are flushed.

***Figure 4.5.3: Activity Diagram — Main Processing Loop***

# Chapter 5: Implementation

## 5.1 Technology Stack

| Technology | Role | Justification |
|---|---|---|
| Python 3.x | Core language | Extensive AI/CV library ecosystem; rapid development |
| YOLOv8 (Ultralytics) | Detection model | State-of-the-art speed and accuracy; clean Python API |
| OpenCV 4.x | Video I/O and rendering | Industry-standard CV library; highly optimized |
| PyTorch 2.x | Deep learning backend | Dynamic computation graph; CUDA GPU acceleration |
| SORT | Object tracking | Lightweight, real-time; no GPU overhead |
| NumPy | Array operations | Efficient numerical computation for bounding box math |
| VS Code | IDE | Extensions for Python, debugging, and Git integration |
| CUDA 11.8 | GPU acceleration | Enables GPU-accelerated inference on RTX 3050 |

***Table 5.1: Technology Stack Summary***

## 5.2 Dataset Collection and Annotation

The custom helmet detection model requires a dedicated annotated dataset. The dataset was assembled from two sources: a publicly available helmet detection dataset from Roboflow Universe (approximately 5,000 images with 'helmet' and 'no_helmet' annotations), supplemented with a smaller set of images collected from industrial safety training materials relevant to the Hindalco environment (approximately 500 images).

Annotations were provided in YOLO format, where each image is paired with a .txt label file. Each line in the label file represents one annotated object in the format: class_index center_x center_y width height, where all coordinates are normalized to the range [0, 1] relative to the image dimensions. Class 0 corresponds to 'helmet' and class 1 corresponds to 'no_helmet'.

Images were resized to 640x640 pixels for training, consistent with YOLOv8's default input resolution. The dataset was split into training (80%), validation (10%), and test (10%)

subsets. Data augmentation was applied during training using YOLOv8's built-in augmentation pipeline, which includes random horizontal flipping, mosaic augmentation, random scaling, HSV color space jittering, and random perspective transforms. These augmentations significantly increase the effective diversity of the training dataset without requiring additional labeled examples.

## 5.3 YOLO Dataset Configuration

The data.yaml configuration file is required by YOLOv8 to locate the dataset and define the class labels. The configuration specifies the paths to the training and validation image directories, the number of classes (nc: 2), and the class names. The YAML file was placed in the project root and referenced directly in the training command.

## 5.4 Model Training

Training was performed using the Ultralytics YOLOv8 Python API. The base model used was yolov8s.pt (Small variant), chosen as a balance between accuracy and training speed given the available hardware. The training configuration used an image size of 640x640, a batch size of 16, and 50 training epochs. The optimizer used was AdamW with a learning rate of 0.01 and cosine learning rate scheduling.

Training was executed on the NVIDIA RTX 3050 GPU using CUDA acceleration. The initial epochs were dominated by rapid loss reduction as the model adapted the pre-trained ImageNet weights to the helmet detection domain. From approximately epoch 15 onwards, the learning slowed to a fine-tuning regime, with gradual improvements in mAP and precision metrics.

YOLOv8's training pipeline automatically saves the best-performing model checkpoint (best.pt) based on the validation mAP50 metric at each epoch. The final best.pt was used for all subsequent inference and evaluation.

## 5.5 Hyperparameter Configuration

| Hyperparameter | Value |
|---|---|
| Model | YOLOv8s (Small) |
| Image Size | 640 x 640 pixels |
| Batch Size | 16 |

| | |
|---|---|
| Epochs | 50 |
| Optimizer | AdamW |
| Learning Rate (initial) | 0.01 |
| LR Scheduler | Cosine Annealing |
| Weight Decay | 0.0005 |
| Confidence Threshold (inference) | 0.40 |
| IoU Threshold (NMS) | 0.45 |
| Warmup Epochs | 3 |
| Mosaic Augmentation | Enabled (p=1.0) |
| Horizontal Flip | Enabled (p=0.5) |
| HSV Augmentation | Enabled |

***Table 5.2: Training Hyperparameters***

## 5.6 Project Folder Structure

The project is organized into a logical directory structure to maintain separation between data, models, source code, and outputs:

```
AI_MultiObject_Detection/ ├── dataset/ │   ├── images/  (train/ val/ test/)
│   ├── labels/  (train/ val/ test/) │   └── data.yaml ├── models/ │   ├──
yolov8n.pt         (COCO pre-trained) │   └── helmet_best.pt    (Custom
trained) ├── runs/ │   └── train/exp/         (Training outputs) ├── src/ │
├── main.py           (Entry point) │   ├── detector.py       (COCO
detection module) │   ├── helmet_detector.py(Helmet detection module) │
├── tracker.py        (SORT tracking module) │   ├── counter.py
(Person counting module) │   └── recorder.py       (Video recording module)
├── output/               (Recorded video files) ├── logs/
(Detection JSON logs) ├── train.py             (Training script) └──
requirements.txt
```

***Figure 5.6.1: Project Directory Structure***

## 5.7 Key Implementation Details

### 5.7.1 Main Detection Loop

The main detection loop (main.py) initializes all modules, opens the video capture, and enters a continuous frame-processing loop. At each iteration, a frame is read; if no frame is returned (end of file or camera disconnection), the loop terminates. The frame is passed to the two detectors concurrently using Python threading to minimize total inference time per frame. Detections from both models are collected, passed to the tracker, and processed by the counter. Annotations are then rendered and the frame is displayed.

### 5.7.2 Bounding Box Rendering

Bounding boxes are rendered on frames using OpenCV's cv2.rectangle() function. The color of the bounding box is determined by the detected class: green (#00FF00) for 'helmet', red (#FF0000) for 'no_helmet', and blue (#0000FF) for general COCO objects. Class labels, confidence scores, and tracking IDs are rendered as text overlays using cv2.putText() with a bold font for visibility. A filled rectangle background is drawn behind each text label to ensure readability against any background.

### 5.7.3 Virtual Line and Counting Logic

The virtual counting line is defined as a set of y-coordinate (for horizontal lines) or x-coordinate (for vertical lines) at a fixed pixel position in the frame. For each tracked person, the centroid (cx, cy) is computed from the bounding box coordinates as ((x1+x2)/2, (y1+y2)/2). The system records the previous centroid position of each tracked ID. A crossing event is registered when the centroid transitions from one side of the line to the other between consecutive frames. A minimum movement threshold is applied to prevent spurious count increments due to bounding box jitter.

## 5.8 Installation and Setup

8. Install Python 3.x from the official Python website and ensure pip is available.
9. Install CUDA 11.8 and cuDNN (compatible with RTX 3050) from the NVIDIA Developer website.
10. Create a virtual environment: python -m venv venv and activate it.
11. Install dependencies: pip install -r requirements.txt (includes ultralytics, opencv-python, torch, torchvision, numpy, filterpy, scipy).
12. Place the pre-trained COCO model (yolov8n.pt) and the custom trained model (helmet_best.pt) in the models/ directory.
13. Configure the dataset paths in dataset/data.yaml if retraining is required.
14. Run the system: python src/main.py --source 0 (for webcam) or python src/main.py --source path/to/video.mp4.

# Chapter 6: Results and Discussion

## 6.1 Training Results

The custom helmet detection model was trained for 50 epochs on the assembled dataset. The training was completed on the NVIDIA RTX 3050 GPU, with each epoch taking approximately 3-4 minutes, giving a total training time of approximately 2.5 to 3 hours. The training and validation loss curves showed a consistent downward trend throughout training, with no evidence of overfitting — the validation loss tracked closely with the training loss, indicating good generalization.

| Model | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| YOLOv8s — Helmet (Epoch 20) | 0.82 | 0.79 | 0.83 | 0.61 |
| YOLOv8s — Helmet (Epoch 35) | 0.88 | 0.85 | 0.89 | 0.67 |
| YOLOv8s — Helmet (Best — Epoch 47) | 0.91 | 0.88 | 0.92 | 0.71 |

***Table 6.1: Helmet Detection Model Training Metrics***

## 6.2 Evaluation Metrics Explained

Precision measures the fraction of positive detections that are true positives. A precision of 0.91 means that 91% of helmet detections made by the model are correct, with only 9% being false positives (non-helmets incorrectly identified as helmets).

Recall measures the fraction of true positive instances that are detected. A recall of 0.88 means the model successfully detects 88% of all helmets actually present in the images, with 12% missed (false negatives).

mAP50 (mean Average Precision at IoU threshold 0.50) is the standard object detection benchmark metric. A detection is counted as correct if its predicted bounding box overlaps with the ground-truth box by at least 50% (IoU >= 0.5). An mAP50 of 0.92 represents excellent performance, exceeding the project target of 0.85.

mAP50-95 averages mAP across IoU thresholds from 0.50 to 0.95 in steps of 0.05, providing a more stringent evaluation of localization quality. A value of 0.71 is competitive for a small model (YOLOv8s) on a two-class dataset.

## 6.3 Runtime Performance

| Metric | GPU (RTX 3050) | CPU Only |
|---|---|---|
| Average FPS | 28-34 FPS | 4-8 FPS |
| Inference Latency (per frame) | ~32 ms | ~180 ms |
| COCO Model (yolov8n) FPS | 35-42 FPS | 6-10 FPS |
| Helmet Model (yolov8s) FPS | 28-34 FPS | 4-8 FPS |
| Combined Pipeline FPS | 22-28 FPS | 3-5 FPS |
| Memory Usage (GPU VRAM) | ~1.8 GB | N/A |
| Memory Usage (RAM) | ~2.1 GB | ~1.6 GB |

***Table 6.2: Runtime Performance Metrics***

## 6.4 Qualitative Results

### 6.4.1 Multi-Object Detection (COCO)

The COCO-trained YOLOv8 model successfully detects persons, vehicles, furniture, electronics, and various other common objects in real-time video from the webcam. Detection is robust across a range of distances and partial occlusions. The model correctly handles multiple instances of the same class (e.g., multiple persons in the frame simultaneously), and confidence scores generally reflect the visual quality of detections — higher confidence for clear, well-lit, frontal views and lower confidence for partially occluded or small-scale objects.

***Figure 6.4.1: COCO Multi-Object Detection — Multiple Classes Detected Simultaneously***

### 6.4.2 Helmet Detection

The custom helmet detection model reliably differentiates between workers wearing helmets (green bounding box) and workers without helmets (red bounding box). In controlled test conditions with industrial-style images from the validation set, the model correctly identifies helmet-wearing status even for partially visible helmet edges and at moderate distances from the camera. The most common failure modes occur when only a small portion of the helmet is visible, or when the helmet colour closely matches the background.

*Figure 6.4.2: Helmet Detection — Green Box (Helmet), Red Box (No Helmet)*

### 6.4.3 Object Tracking and Person Counting

The SORT tracker successfully maintains unique IDs across frames for persons moving through the scene at natural walking speeds. Track loss and reassignment occasionally occurs during sudden direction changes or when persons briefly exit and re-enter the frame. The person counting mechanism accurately registers crossings of the virtual line under normal conditions. False counts occasionally occur when a tracked person pauses near the counting line and the bounding box centroid oscillates across it due to detection jitter.

*Figure 6.4.3: Object Tracking with Unique IDs and Person Counting Line*

## 6.5 Discussion

The results demonstrate that the AI Multi-Object Detection System achieves its core objectives: reliable real-time detection, effective helmet compliance monitoring, robust tracking, and accurate person counting. The GPU-accelerated pipeline delivers frame rates well above the minimum threshold required for smooth real-time monitoring, making it suitable for live deployment on CCTV-connected workstations.

The mAP50 of 0.92 for helmet detection represents a significant achievement, particularly given the relatively modest size of the training dataset (~5,500 images). This performance level is attributable to the strong feature representations already encoded in the YOLOv8s pre-trained weights (from ImageNet and COCO pre-training), combined with effective data augmentation during fine-tuning.

The primary limitations of the system are its dependency on adequate lighting conditions, the fixed nature of the virtual counting line, and occasional tracking failures during rapid motion or occlusion. These limitations are inherent to the current architecture and hardware setup and can be addressed through the improvements outlined in the Future Scope chapter.

# Chapter 7: Advantages and Limitations

## 7.1 Advantages

- High-speed real-time detection: GPU-accelerated YOLOv8 inference achieves 28-34 FPS on the RTX 3050, enabling smooth real-time monitoring without perceptible latency.
- Comprehensive multi-object awareness: The COCO-trained model simultaneously detects 80 object categories, providing broad situational awareness beyond just helmet compliance.
- Accurate helmet compliance monitoring: The custom-trained model achieves 92% mAP50 on helmet detection, providing a reliable automated safety compliance tool.
- Persistent object tracking: SORT-based tracking maintains unique identities across frames, enabling individual-level monitoring rather than just instantaneous detection.
- Access control support: The virtual person counting line provides an automated mechanism for monitoring entries and exits through designated checkpoints.
- Evidence documentation: Video recording capability creates a timestamped audit trail of all detection events for compliance reporting and incident investigation.
- Scalability: The modular architecture allows easy integration of additional detection models, camera feeds, and analytical capabilities without redesigning the core pipeline.
- Cost-effectiveness: The system leverages open-source software and existing camera infrastructure, requiring no specialized proprietary hardware beyond a capable GPU.
- GPU acceleration: CUDA-enabled PyTorch inference dramatically reduces latency compared to CPU-only alternatives, making real-time operation practical on mid-range consumer hardware.

## 7.2 Limitations

- Lighting dependency: Detection accuracy degrades in low-light conditions, requiring adequate ambient illumination or supplemental IR lighting for nighttime deployment.
- Camera quality sensitivity: Low-resolution or motion-blurred video feeds reduce detection reliability, particularly for smaller objects such as helmets at distance.
- Dataset-specific model performance: The custom helmet model may require retraining or fine-tuning if deployed in environments with substantially different helmet designs or background conditions.

- Fixed counting line: The virtual counting line is statically configured and requires manual adjustment if the camera angle or installation position changes.
- Occlusion handling: Heavily occluded objects may be missed or incorrectly detected; tracking reliability drops when persons fully exit the frame or are blocked by obstacles.
- Single-camera architecture: The current implementation processes a single video stream; multi-camera integration requires additional development.
- No automated alerting: The system currently provides only visual output; integration with alert notification systems (SMS, email, alarm) is a future enhancement.
- GPU requirement for real-time operation: Satisfactory frame rates require a CUDA-compatible GPU; CPU-only deployment results in frame rates insufficient for smooth real-time monitoring.

# Chapter 8: Future Scope

The foundations established by this project open numerous avenues for future enhancement and expansion. The following directions represent the most impactful opportunities for extending the system's capabilities:

## 8.1 Multi-Camera Network Integration

The current system processes a single video stream. Production deployment in a large industrial facility would require simultaneous processing of multiple CCTV feeds. A future version could implement a multi-stream processing pipeline using multiprocessing or distributed computing frameworks such as Apache Kafka for stream management. A central monitoring dashboard would aggregate detection events from all cameras, providing facility-wide safety oversight.

## 8.2 Cloud Dashboard and Remote Monitoring

Integrating the system with a cloud-based monitoring dashboard (e.g., built with React, Node.js, and a time-series database) would enable remote access to real-time detection feeds, historical analytics, and compliance reports. Alert notifications via SMS, email, or messaging platforms could be automatically triggered when non-compliance events (e.g., no_helmet detected) occur.

## 8.3 Extended PPE Detection

Beyond helmets, industrial safety protocols require compliance with additional PPE items including safety vests (high-visibility jackets), safety gloves, safety goggles, and steel-toed footwear. A future iteration of the system could incorporate detection models for each of these PPE categories, providing comprehensive, simultaneous multi-class PPE compliance monitoring.

## 8.4 Face Recognition and Identity Management

Integration of face recognition using libraries such as DeepFace or the FaceNet architecture would enable the system to identify individual workers by name or employee ID. This would allow the system to associate safety violations with specific individuals and generate personalized compliance reports, significantly enhancing accountability.

## 8.5 Number Plate Recognition

Extending the system with Automatic Number Plate Recognition (ANPR) using a custom-trained detection and OCR pipeline would enable monitoring and logging of vehicle movements within the industrial facility, complementing the person-tracking capabilities.

## 8.6 Anomaly Detection and Behavioural Analysis

Beyond classification-based detection, future versions could incorporate anomaly detection models trained to identify unusual behaviours such as workers entering restricted zones, individuals falling, or large crowds gathering unexpectedly near hazardous equipment. Skeleton-based pose estimation using MediaPipe or OpenPose could enable detection of hazardous postures or fall events.

## 8.7 Model Optimization for Edge Deployment

For deployment on low-power edge devices such as NVIDIA Jetson Nano or Raspberry Pi with AI accelerators, model quantization (INT8 precision via TensorRT) and pruning techniques could be applied to reduce model size and inference latency without significant accuracy loss. This would enable distributed deployment of detection nodes at individual camera locations, reducing bandwidth requirements for central processing.

## 8.8 Continuous Learning Pipeline

A continuous learning pipeline could be implemented to automatically collect hard examples (images where the model is uncertain or incorrect) from the production deployment and route them for human annotation and periodic model retraining. This would enable the model to progressively improve its accuracy in the specific deployment environment over time.

# Chapter 9: Summary and Conclusion

## 9.1 Summary

This internship project, undertaken at Hindalco Industries Limited from June to July 2026, developed a comprehensive AI Multi-Object Detection System using Python, OpenCV, YOLOv8, and PyTorch. The system integrates four core technical capabilities into a unified real-time pipeline: multi-class object detection using a pre-trained COCO model, safety helmet compliance detection using a custom-trained model, object tracking using the SORT algorithm, and person counting using a virtual line mechanism.

The custom helmet detection model was trained from scratch on a dataset of approximately 5,500 annotated images using the YOLOv8s architecture, achieving a best validation mAP50 of 0.92 and a Precision of 0.91 — exceeding the project's target performance threshold. The system achieved GPU-accelerated real-time inference at 22-28 FPS for the combined detection pipeline on the NVIDIA RTX 3050, comfortably satisfying the requirements for smooth live monitoring.

The video recording module enables timestamped archiving of annotated output for post-analysis and compliance documentation. The JSON detection log provides a structured audit trail of all detection events. The modular codebase is organized for extensibility, enabling straightforward addition of new detection models, camera sources, and analytical modules.

## 9.2 Conclusion

The AI Multi-Object Detection System demonstrates that state-of-the-art computer vision and deep learning technologies can be effectively applied to address real-world industrial safety challenges with practical, accessible tools and hardware. The project validates that YOLOv8, combined with SORT tracking and a Python-OpenCV pipeline, provides a compelling foundation for industrial video analytics applications.

From an academic perspective, this internship has provided deep, hands-on experience across the full machine learning lifecycle: dataset collection, annotation, training, hyperparameter tuning, evaluation, and deployment. The exposure to a real industrial environment at Hindalco Industries Limited has contextualized these technical skills within a concrete, safety-critical application domain.

The system as developed is a functional prototype ready for controlled pilot deployment. With the enhancements outlined in the Future Scope chapter — particularly multi-camera integration, cloud dashboard connectivity, and automated alerting — it has the potential to serve as a production-grade safety monitoring solution. The low-cost, open-source nature of the technology stack makes it accessible to a wide range of industrial organizations beyond large enterprises.

In summary, this project successfully demonstrates the application of artificial intelligence and computer vision to industrial safety monitoring, delivering a technically sound, practically relevant, and extensible system that meets all defined objectives and exceeds key performance targets.

## 9.3 Learning Outcomes

This internship provided the following key learning outcomes:

- Developed proficiency in training, evaluating, and deploying custom YOLOv8 object detection models.
- Gained hands-on experience with the complete dataset pipeline: collection, annotation in YOLO format, augmentation, and train/val/test splitting.
- Implemented and integrated the SORT multi-object tracking algorithm with a real-time detection pipeline.
- Applied OpenCV for professional-quality video processing, annotation, and recording.
- Understood the practical impact of hardware acceleration on real-time AI system performance.
- Developed appreciation for the engineering challenges of deploying AI in industrial safety contexts.
- Strengthened proficiency in Python software engineering practices including modular architecture and logging.

# Appendix

## Appendix A: Training Script (train.py)

The training script initializes the YOLOv8 model from the pre-trained small checkpoint (yolov8s.pt) and invokes the training loop via the Ultralytics model.train() API. Key parameters passed include the path to the data.yaml configuration, image size (640), batch size (16), number of epochs (50), and the device flag set to '0' for the first available CUDA GPU. The model.train() call handles all aspects of the training loop including gradient computation, weight updates, metric logging, and checkpoint saving. Upon completion, training metrics are saved to the runs/train/exp/ directory and the best model checkpoint (best.pt) is copied to the models/ directory for use in inference.

## Appendix B: Requirements File (requirements.txt)

```
ultralytics>=8.0.0 opencv-python>=4.7.0 torch>=2.0.0 torchvision>=0.15.0
numpy>=1.24.0 matplotlib>=3.7.0 scipy>=1.10.0 filterpy>=1.4.5 pyyaml>=6.0
Pillow>=9.5.0 tqdm>=4.65.0
```

## Appendix C: data.yaml Configuration

```
# Helmet Detection Dataset Configuration path: ./dataset train: images/train
val:   images/val test:  images/test  nc: 2  # Number of classes names:   0:
helmet   1: no_helmet
```

## Appendix D: Keyboard Controls During Runtime

| Key | Action |
|---|---|
| Q | Quit the application and release all resources |
| R | Toggle video recording on/off |
| H | Toggle display of helmet detection overlay |
| C | Toggle display of COCO detection overlay |
| T | Toggle display of tracking IDs |
| L | Toggle display of virtual counting line |
| S | Save current frame as a screenshot |
| SPACE | Pause/resume the video feed |

***Table D.1: Keyboard Controls***

# BIBLIOGRAPHY

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779-788.

[2] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in Proc. IEEE CVPR, 2017, pp. 7263-7271.

[3] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv:2004.10934, 2020.

[5] G. Jocher et al., "ultralytics/yolov5: v6.0," Zenodo, 2020. [Online]. Available: https://github.com/ultralytics/yolov5

[6] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," in Proc. IEEE CVPR, 2023.

[7] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in Proc. IEEE ICIP, 2016, pp. 3464-3468.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770-778.

[10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.

[12] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, 2014, pp. 740-755.

[13] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proc. IEEE CVPR, 2014.

[14] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE TPAMI, vol. 39, no. 6, pp. 1137-1149, 2017.

[15] W. Liu et al., "SSD: Single Shot MultiBox Detector," in Proc. ECCV, 2016, pp. 21-37.

[16] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in Proc. IEEE ICCV, 2017, pp. 2961-2969.

[17] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in NeurIPS, vol. 32, 2019.

[18] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.

[19] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO Series in 2021," arXiv:2107.08430, 2021.

[20] Ultralytics, "YOLOv8 Documentation," 2023. [Online]. Available: https://docs.ultralytics.com

[21] OpenCV Contributors, "OpenCV Documentation," 2024. [Online]. Available: https://docs.opencv.org

[22] PyTorch Contributors, "PyTorch Documentation," 2024. [Online]. Available: https://pytorch.org/docs

[23] NVIDIA Corporation, "CUDA Programming Guide," 2023. [Online]. Available: https://docs.nvidia.com/cuda/

[24] Roboflow, "Helmet Detection Dataset," Roboflow Universe, 2023. [Online]. Available: https://universe.roboflow.com

[25] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.